Fixing the Stale-Locator Race Condition in PrimeNG Dropdown Automation

Step-by-step remediation plan for Selenium + Cucumber automation on the Commodity Contract Management (CCM) module

Prepared for: Girish Kumar M R — QA Automation Engineer, Invensoft
Project: ERP-Automation — Commodity Contract Management (CCM)
Stack: Selenium WebDriver · Java · Cucumber BDD · Page Object Model · TestNG

Root cause	Wrong ExpectedConditions overload — waiting on a cached WebElement reference instead of a live By
Symptom	TimeoutException: element not visible, intermittent — only on fields affected by re-render or scroll clipp
Confirmed affected	selectTableDropdownValue() — Packing dropdown
At risk, not yet failing	selectClassDropdown() and any other dropdown using the WebElement overload
Fix strategy	By-based waits + generic PrimeNG dropdown helper + horizontal scroll handling + panel-close wait

Contents

1. Diagnosis Recap

Why It Works for Product and Units but Fails on Packing

The failing call passed an already-located `WebElement` into `ExpectedConditions.elementToBeClickable(...)` instead of a `By` locator. Selenium then polled that one frozen DOM reference for 40 seconds without ever re-querying the page. This is not a locator problem — the XPath is fine. It is a **stale-reference race condition** that only manifests when the underlying node is re-created or scrolled out of view between the moment it was located and the moment the wait resolves.

Why each field behaved differently:

Field	Re-render risk	Scroll-clipping risk	Result
Product (1st column)	None — first interaction, nothing has changed yet	Low — leftmost column, always in initial viewport	Passes
Units (2nd column, plain input)	Low — static input, no overlay/cascade	Low — usually within initial viewport	Passes
Packing (3rd column)	Possible — may be affected by Product/Units cascade	Possible — later columns likely scrolled out	Fails on smaller viewports
dropdown_Class (standalone field)	Currently none — not wired to any cascading logic	Low — standalone form field, not inside the grid	Passes today, at risk later

Key takeaway: this is a latent bug present in every call site using the WebElement overload. Fields that currently pass are not "safe" — they simply haven't hit the timing condition that exposes the bug yet. The fix must be applied everywhere the pattern is used, not only to Packing.

2. Step 1 — Upgrade WebDriverCommonLibrary

Add By-based overloads alongside the existing WebElement-based method. Nothing that currently passes will break — you migrate call sites incrementally in Step 6.

```
// utils/WebDriverCommonLibrary.java

public class WebDriverCommonLibrary {

    private static final Duration DEFAULT_TIMEOUT = Duration.ofSeconds(40);

    // EXISTING - keep for now, mark deprecated
    @Deprecated // Can wait on a stale WebElement reference - see migration plan
    public WebElement explicitWait(WebDriver driver, WebElement element) {
        WebDriverWait wait = new WebDriverWait(driver, DEFAULT_TIMEOUT);
        return wait.until(ExpectedConditions.elementToBeClickable(element));
    }

    // NEW #1 - re-queries the DOM on every poll, self-heals against re-renders
    public WebElement explicitWait(WebDriver driver, By locator) {
        WebDriverWait wait = new WebDriverWait(driver, DEFAULT_TIMEOUT);
        return wait.until(ExpectedConditions.elementToBeClickable(locator));
    }

    // NEW #2 - confirms something (e.g. a PrimeNG panel) has fully closed
    public void explicitWaitForInvisibility(WebDriver driver, By locator) {
        WebDriverWait wait = new WebDriverWait(driver, DEFAULT_TIMEOUT);
        wait.until(ExpectedConditions.invisibilityOfElementLocated(locator));
    }

    // NEW #3 - presence only, used before scrollIntoView
```

```
    public WebElement explicitWaitForPresence(WebDriver driver, By locator) {
        WebDriverWait wait = new WebDriverWait(driver, DEFAULT_TIMEOUT);
        return wait.until(ExpectedConditions.presenceOfElementLocated(locator));
    }

    // NEW #4 - presence + horizontal scroll-into-view + fresh clickable re-check
    public WebElement explicitWaitWithScroll(WebDriver driver, By locator) {
        WebElement el = explicitWaitForPresence(driver, locator);
        ((JavascriptExecutor) driver).executeScript(
            "arguments[0].scrollIntoView({block: 'nearest', inline: 'center'});", el
        );
        return explicitWait(driver, locator); // re-locate fresh, don't reuse 'el'
    }
}
```

Why this fixes it: when Selenium is given a `By` locator, `ExpectedConditions` re-runs `driver.findElement(...)` on every 500ms poll cycle. If Angular destroys and recreates the node mid-wait, the next poll picks up the new node automatically instead of staring at an orphaned reference.

3. Step 2 — Generic PrimeNG Dropdown Helper

One method every dropdown interaction in the app should route through — filterable or not, inside the CCM grid or a standalone field.

```
// utils/PrimeNgDropdownHelper.java

public class PrimeNgDropdownHelper {

    private final WebDriver driver;
    private final WebDriverCommonLibrary lib;

    private static final By PANEL = By.xpath(
        "//div[contains(@class,'p-dropdown-panel')]");
    private static final By PANEL_VISIBLE = By.xpath(
        "//div[contains(@class,'p-dropdown-panel') and " +
        "not(contains(@style,'display: none'))]");
    private static final By FILTER_INPUT = By.xpath(
        "//div[contains(@class,'p-dropdown-panel')]" +
        "//input[contains(@class,'p-dropdown-filter')]");

    public PrimeNgDropdownHelper(WebDriver driver, WebDriverCommonLibrary lib) {
        this.driver = driver;
        this.lib = lib;
    }

    /**
     * Generic entry point for ANY PrimeNG dropdown.
     * @param triggerLocator locator for this dropdown's trigger
     *        (span[@role='combobox'])
     * @param optionText     visible text of the option to select
     * @param hasFilter      whether this dropdown has a search/filter input
     */
    public void select(By triggerLocator, String optionText, boolean hasFilter) {
        // Step 0: defensive - close any leftover open panel
        ensureNoOpenPanel();

        // Step 1: wait + scroll + click trigger
        WebElement trigger = lib.explicitWaitWithScroll(driver, triggerLocator);
        trigger.click();

        // Step 2: wait for panel to attach
        lib.explicitWait(driver, PANEL);

        // Step 3: filter if applicable
        if (hasFilter) {
            WebElement filterInput = lib.explicitWait(driver, FILTER_INPUT);
            filterInput.clear();
            filterInput.sendKeys(optionText);
        }

        // Step 4: locate + click the option (re-queried fresh via By)
        By optionLocator = By.xpath(String.format(
            "//div[contains(@class,'p-dropdown-panel')]" +
            "//li[contains(@class,'p-dropdown-item') and " +
            "not(contains(@class,'p-dropdown-item-group'))]" +
            "[normalize-space(.)='%s']",
            optionText
        ));
        WebElement option = lib.explicitWait(driver, optionLocator);
        option.click();

        // Step 5: wait for panel to fully close before returning
        lib.explicitWaitForInvisibility(driver, PANEL);
    }

    /** Defensive cleanup: close a panel left open by a prior step. */
```

```
    public void ensureNoOpenPanel() {
        List<WebElement> openPanels = driver.findElements(PANEL_VISIBLE);
        if (!openPanels.isEmpty()) {
            new Actions(driver).sendKeys(Keys.ESCAPE).perform();
            lib.explicitWaitForInvisibility(driver, PANEL_VISIBLE);
        }
    }
}
```

4. Step 3 — Grid Column / Row Resolver

Separate from the interaction helper above — this class only computes *which* <td> a dropdown lives in in the Product Details grid, using header text and preceding-sibling counting, index-free and resilient to column reordering.

```
// utils/GridColumnResolver.java

public class GridColumnResolver {

    private final WebDriver driver;

    public GridColumnResolver(WebDriver driver) {
        this.driver = driver;
    }

    public By getTriggerLocator(String sectionHeading, String columnHeader,
                                int rowIndex) {
        return By.xpath(String.format(
            "//h3[normalize-space()='%s']" +
            "/ancestor::*[self::div][.//table][1]" +
            "//table[1]/tbody/tr[%d]/td[" +
              "count(" +
                "//h3[normalize-space()='%s']" +
                "/ancestor::*[self::div][.//table][1]" +
                "//table[1]/thead/tr[1]/th[./span[normalize-space(.)='%s']]" +
                "/preceding-sibling::th" +
              ") + 1" +
            "]//xbs-dcloud-dropdown-group//span[@role='combobox']",
            sectionHeading, rowIndex, sectionHeading, columnHeader
        ));
    }
}
```

Note: the `xbs-dcloud-dropdown-group` *wrapper in the final step is a type-guard — it makes the locator fail loudly if it ever lands on a non-dropdown cell (e.g. a number input) rather than silently clicking the wrong control.*

---

5. Step 4 — Rewire the Page Object

```
// pages/CommodityContractManagementPage.java

public class CommodityContractManagementPage {

    private final WebDriver driver;
    private final PrimeNgDropdownHelper dropdownHelper;
    private final GridColumnResolver resolver;

    public CommodityContractManagementPage(WebDriver driver) {
        this.driver = driver;
        WebDriverCommonLibrary lib = new WebDriverCommonLibrary();
        this.dropdownHelper = new PrimeNgDropdownHelper(driver, lib);
        this.resolver = new GridColumnResolver(driver);
    }

    /** Replaces selectTableDropdownValue - generic for ANY grid column. */
    public void selectTableDropdownValue(String sectionHeading, String columnHeader,
            int rowIndex, String optionText, boolean hasFilter) {
        By trigger = resolver.getTriggerLocator(sectionHeading, columnHeader, rowIndex);
        dropdownHelper.select(trigger, optionText, hasFilter);
    }
```

```
    /** Replaces selectClassDropdown - for standalone (non-grid) dropdowns. */
    public void selectClassDropdown(String optionText) {
        By trigger = By.xpath(
            "//span[@formcontrolname='class' or contains(@id,'class')]" +
            "//span[@role='combobox']"); // replace with actual Class field locator
        dropdownHelper.select(trigger, optionText, true);
    }
}
```

6. Step 5 — Update Step Definitions

```
// steps/StepsContainer.java

@When("user selects {string} from the {string} dropdown in the {string} table")
public void userSelectsFromTheDropdownInTheTable(String optionText,
        String columnHeader, String sectionHeading) {
    commodityContractManagementPage.selectTableDropdownValue(
        sectionHeading, columnHeader, 1, optionText, true
        // adjust hasFilter per column if needed
    );
}
```

7. Step 6 — Migration Order for Existing Methods

Migrate field-by-field, testing after each change rather than converting the whole framework in one pass.

Order	Field / Method	Why this order
1	selectTableDropdownValue (Packing)	Confirmed broken — fix and verify first
2	Other grid dropdowns (UOM, Target Type, Trade Month)	Same resolver, low risk once Step 1 passes
3	selectClassDropdown	Not broken yet, but same latent bug — fix proactively
4	Remaining standalone dropdowns elsewhere in the app	Lowest urgency, identical pattern

Don't touch Product/Units unless they turn out to be at risk too — Product is a dropdown (candidate for the same helper), Units is a plain number input and needs a separate p-inputnumber helper if you choose to build one.

---

8. Step 7 — Add a Regression Test for the Race Condition

A single green run does not prove a race condition is fixed. Add a scenario that exercises the exact cascade sequence, and run it several times in a row.

```
# CreateCommodityContractManagement.feature

Scenario: Selecting Product does not break subsequent Packing selection
  Given user is on the Create Commodity Contract Management page
  When user selects "Wheat" from the "Product" dropdown in the "Product Details" table
  And user selects "Bags" from the "Packing" dropdown in the "Product Details" table
  Then the "Packing" field should show "Bags"
```

Run this scenario 3–5 consecutive times (rerun manually, or loop it in your CI job) before considering the fix verified — flaky/race-condition bugs can pass on a single run by chance.

9. Step 8 — Retire the Old Overload

Once every dropdown call site uses the By-based overload, remove the deprecated WebElement-based method so the fragile pattern can't silently creep back in via copy-paste into a new method later.

```
// Search the codebase for: explicitWait(driver,
// Confirm every call site passes a By, not a WebElement, then delete:

@Deprecated
public WebElement explicitWait(WebDriver driver, WebElement element) { ... }
```

10. Final Sanity Checklist

- WebDriverCommonLibrary has By-based explicitWait, explicitWaitForInvisibility, and explicitWaitWithScroll
- PrimeNgDropdownHelper.select(...) is the single method every dropdown interaction routes through
- GridColumnResolver computes column position dynamically — never a hardcoded td[n]
- Every dropdown selection ends with a panel-invisibility wait, not just a click
- Packing scenario passes 3–5 consecutive runs, not just once
- Old WebElement-based explicitWait overload is migrated everywhere or removed

Suggested Next Extension

The same generic-helper pattern applies to the other field types in the Product Details grid: a p-inputnumber helper for Qty/Price, and a p-calendar / date-range helper for Target Date Range. Building those on top of the same GridColumnResolver keeps every field in the row covered by consistent, reusable, DOM-re-render-safe methods.